

CLAUDE CERTIFIED DEVELOPER FOUNDATIONS

What to know for the exam.

Full course:

By Jacob Bushong

EXAM FORMAT

- ▶ 53 scenario-based questions
 - ▶ 120 minutes
 - ▶ 720 / 1000 to pass
 - ▶ Proctored via Pearson VUE
-

Domain 1: Agents and Workflows

Exam weight: 15%

Section 1A — Agent Architecture

Workflow vs. Agent: Decision Criteria and Tradeoffs *(Lessons 1 and 2)*

The dividing line between a workflow and an agent, and the cost, latency, and reliability math that decides which one a task actually needs.

- ▶ A workflow fixes control flow at design time in your code; an agent lets the model decide the next action at runtime in a reason-act-observe loop. Tool use is not the distinction, since both use tools; who owns the next decision is the line.
- ▶ The shared building block is the augmented LLM: a model plus retrieval, tools, and memory. Anthropic's guidance is to use the simplest thing that works, and many tasks are solved by one call with retrieval and a couple of examples.
- ▶ Three signals point to an agent, and you usually want more than one: an open-ended space you can't enumerate, an unknown number of steps, and value in letting the model choose which tool and when. The three deterministic workflow shapes are prompt chaining, routing, and parallelization.
- ▶ An agent trades latency and token cost for task performance on three axes: a loop makes sequential calls so users wait longer, every turn re-sends the growing context so the bill climbs (large tool outputs are the biggest driver), and adaptability is what you buy with the other two.
- ▶ Compounding errors are the sneakiest cost: a wrong early step becomes input the model treats as fact, so longer autonomous runs carry more risk. Workflows give predictability, auditability, and consistency, so validate the workflow first and reach for a hybrid that drops a scoped agent into just the one open-ended stage.

Manager/Supervisor Hierarchies and the Role of Subagents *(Lessons 3 and 4)*

How a manager decomposes a goal across scoped subagents, and why the context isolation each subagent runs in is the point of delegating, not a limitation.

- ▶ The manager runs one repeatable loop, decompose, delegate, synthesize, and never does the subtask work itself. Orchestrator-workers differ from fixed parallelization by who decides the subtasks: the manager decides at runtime, parallel code splits them ahead of time.
- ▶ Delegation happens through the built-in Agent tool; each subagent carries a description Claude matches against the task, and you can also name one directly. Allow-list the Agent tool to auto-approve the delegation, but the subagent's own tool calls still go through permissions separately.
- ▶ Every subagent starts in a fresh, isolated context. It inherits only its own prompt, project CLAUDE.md, and tools, plus whatever you pass in the task; it never sees the manager's history, tool results, or system prompt. If it needs a fact, you must hand it over.
- ▶ Only the subagent's final message returns to the manager, so intermediate reasoning and tool output stay contained. That bounds token growth predictably as subtasks scale and delivers the three payoffs: context isolation, parallelization, and tool restriction.
- ▶ Three guardrails keep delegation safe and all live at the manager: least-privilege tools and permissions, a side-effect boundary (no writes or actions outside the assigned subtask), and result validation before synthesis. The three anti-patterns are overly broad scope, underpowered context, and treating subagents as only a parallel-speed trick.

Section 1B — Agent Construction with Claude

Claude Agent SDK: `query()`, the Agent Loop, and Input Modes *(Lessons 5 to 7)*

The Claude Agent SDK's `query()` entry point, the agent loop it runs, and the two input modes that feed prompts into that one shared loop.

- ▶ The Agent SDK packages Claude Code's tools, agent loop, and context management for Python and TypeScript and is standalone, so no CLI install is needed. `query()` is the primary entry point, opens a fresh session with no memory by default, and returns an async stream you iterate. It shares one loop with `ClaudeSDKClient`; the client only adds a persistent session and mid-run interrupts.
- ▶ `query()` options set the system prompt, tools, model, and `max_turns`. Install with `pip install claude-agent-sdk` (Python 3.10+) or the npm package, and authenticate with `ANTHROPIC_API_KEY` or an env flag for Bedrock or Vertex.
- ▶ The loop runs five steps: Claude receives the prompt and an init message opens the run, Claude responds with text or tool calls, the SDK executes tools and feeds results back, the turn repeats if more tools are called, and the loop returns once Claude replies with no tool. One turn is a single tool round trip.
- ▶ By default a run has no cap; `max_turns` limits tool-using turns and `max_budget_usd` caps spend (checked between calls, so it can slightly overshoot). Setting a bound is the developer's job, and an unbounded loop is a defect. The `ResultMessage` carries `subtype`, `total_cost_usd`, `usage`, `num_turns`, and `session_id`, but only a success subtype carries the final result text, so branch on subtype first.
- ▶ Streaming input is the default and recommended mode: an async series of messages, a persistent process, and support for images, queued messages, live interrupts (via the client), and natural multi-turn context. Single-message input is one string answered once, best for stateless or serverless jobs. Input streaming (sending prompts) is separate from output streaming, which you turn on with `include_partial_messages`, and both differ from the raw API stream flag.

Custom Agent Loops and Harnesses *(Lessons 8 and 9)*

Dropping below the Agent SDK to hand-roll an agent loop against the Messages API, and the harness of turn accounting, state, and stop conditions that makes a raw loop safe to run.

- ▶ There are three levels of control: the Messages API (the raw endpoint, best for custom loops), the Agent SDK (Claude Code as a library that runs the loop for you), and Managed Agents (Anthropic hosts the loop and sandbox). A custom loop trades convenience for owning tool execution, retries, and logging, at the cost of more code you maintain.
- ▶ The custom-loop cycle is fixed: send the messages array, detect `tool_use` blocks, execute each tool, append a `tool_result` tied to its `tool_use` by matching id, then re-invoke. Miss or mismatch the `tool_result` and the very next call fails.
- ▶ Inspect `stop_reason` every turn: `tool_use` means Claude paused for your code to run the tool and continue (Anthropic runs nothing), while `end_turn` means exit with the answer. Client tools run in your app and need a `tool_result`; server tools like web search and code execution run on Anthropic infrastructure and return results directly.
- ▶ The harness is the loop plus its infrastructure and owns four jobs: turn accounting, state tracking, error handling (retry, skip, or abort), and explicit stop conditions. The turn budget must be enforced in your runtime, never left to the model to self-limit, because an unbounded loop is a defect.
- ▶ Owning the loop gives three seams to inject policy: before a tool runs (check, log, or block, where deterministic policy lives), around execution (retries and timeouts), and after the result (log and record the turn count). When streaming, tool arguments arrive as `input_json_delta` fragments, so accumulate the partial JSON and parse only when the block closes.

Managed Agent Deployment: Self-Hosted vs. Anthropic-Hosted *(Lessons 10 and 11)*

Where a Managed Agent's tools physically run across the Anthropic-hosted and self-hosted shapes, and the five-piece agent definition that stays fixed no matter which shape you deploy.

- ▶ Managed Agents is a hosted service where Anthropic runs the agent loop and sessions via a REST API. You choose the environment, meaning where a session's tools execute; the control plane, where Claude reasons and the loop turns, always stays with Anthropic.
- ▶ Anthropic-hosted is the default: tools and code run in Anthropic's managed cloud sandboxes, one per session, with Anthropic owning egress, file mounting, and lifecycle, so there is no infrastructure for you to operate. Self-hosted moves tool execution onto your infrastructure via the `self_hosted` environment type, keeping code, filesystem, and egress in your environment.
- ▶ Self-hosted relocates tool execution only, not the model or the loop. Tool inputs and outputs still flow to Anthropic's control plane so the model can act, so self-hosting is not an air gap by itself. Self-host for data residency and compliance, to reach private non-publicly-routable services, or for end-to-end audit and security control.
- ▶ An agent is defined by five pieces, model, system prompt, tools, MCP servers, and skills, created once and referenced by ID across sessions. The model and system prompt are the core; tools, MCP servers, and skills extend it. A session is a single running instance, and many sessions can point at one agent.
- ▶ The model is set on the agent, not the environment, and self-hosted supports every model Managed Agents offers, so switching hosting never forces switching models. The same agent ID deploys

Anthropic-hosted or self-hosted unchanged, which is why the common path is to prototype hosted, then graduate to self-hosted by changing only the environment.

Hooks for Deterministic Actions *(Lessons 12 and 13)*

How hooks run your own code at fixed lifecycle events to enforce deterministic control the model cannot argue with, and how a PreToolUse deny becomes a guardrail the prompt can't undo.

- ▶ A hook is a callback the runtime invokes automatically at a lifecycle event; it runs in your application process, not the model's context window, so it costs no context tokens and the model cannot be talked out of it firing. The five events are PreToolUse, PostToolUse, Notification, Stop, and SessionStart (a settings-file shell hook, not a Python callback).
- ▶ PreToolUse fires before a tool runs, so only it can block the call, rewrite its input, or require approval; PostToolUse fires after, so it can only log or react and cannot stop an action already taken. To prevent harm the check must live in PreToolUse; putting it in PostToolUse or Stop quietly fails after the damage is done.
- ▶ A PreToolUse hook sets permissionDecision to one of four values: allow (an empty result means the same), deny (the model sees your reason and moves on), ask (pause and show the user), or defer (end the query for later, and any rewritten input is ignored). When hooks conflict, precedence is deny over defer over ask over allow.
- ▶ The guardrail pattern is small: a matcher selects which tool calls to watch, the hook inspects the pending input for a dangerous pattern, and it returns deny before execution. Hooks register via options.hooks in code or as shell hooks in settings.json (loaded through setting_sources), where a command hook returns exit code 2 to block at the shell level.
- ▶ Because the check lives in code outside the model's reach, no input can jailbreak it and it fires the same way every run, which is why real guardrails are hooks, not prompt text. Hooks pair with least privilege: permissions set the baseline of what is possible, and hooks add deterministic case-by-case control to block, gate for human approval, or audit.

Section 1C — Agent Patterns and Frameworks

Agent Design Patterns: Tool-Use Loops, Subagents, Memory, Context *(Lessons 14 and 15)*

The tool-use loop and subagents that let an agent act and stay organized, and the active discipline of managing a context window that only ever accumulates.

- ▶ In the tool-use loop the model picks a tool and its input, the runtime runs it, and the observation feeds back so the model chooses its next move; the loop ends at a text-only reply. Read-only tools like reading, searching, or querying have no side effects and run concurrently, while state-modifying tools run sequentially, and custom tools default to sequential unless marked read-only.
- ▶ A subagent handles a focused subtask in its own conversation, so intermediate reasoning and tool output stay inside it and only its final message returns to the parent. The four reasons to delegate are context isolation, parallelization (a batch finishes in the time of the slowest), tool restriction, and specialized prompts. A subagent starts fresh with no shared context, so you must pass in everything it needs.

- ▶ The context window is the token budget the model attends to in one request, spanning the system prompt, history, tool definitions, and tool results. It does not reset between turns; everything accumulates and nothing drops unless your application removes it, and large tool outputs are the biggest driver of bloat.
- ▶ In-context memory is fast but ephemeral, gone when the window truncates or the session ends; external memory is durable, surviving compaction and new sessions, so long-lived data belongs there. Reaching the window's limit is not a place to store durable facts. Reclaim tokens by pruning stale tool output, compacting older history, or starting a fresh session re-grounded from memory.
- ▶ Some management is automatic: the runtime compacts old history near the limit (early detail can be lost), and prompt caching reuses stable prefixes to cut cost and latency but persists nothing. Instructions in CLAUDE.md are re-injected on every request and survive compaction, which is exactly why durable behavior rules belong there.

Agentic Frameworks: LangGraph vs. Strands vs. PydanticAI *(Lessons 16 to 18)*

The identities of LangGraph, Strands, and PydanticAI, matching each to the build it fits, and the cross-attribution trap of assigning one framework's primitives to another.

- ▶ LangGraph is the graph-based framework: a StateGraph builder over a typed State schema (usually a TypedDict) that every node reads. Nodes are functions returning a partial update of only the keys they touch, never the whole state or the next node; edges own routing (static add_edge or conditional add_conditional_edges), and you must compile() before running.
- ▶ In LangGraph, with no reducer a node's update overwrites the key; attach one with Annotated (like add) to append, and add_messages appends chat messages and updates by ID. A checkpointer is thread-scoped short-term persistence saving a snapshot at every super-step (powering continuity, human-in-the-loop, and time-travel), while the separate Store holds cross-thread long-term memory outside graph state; do not reverse the two.
- ▶ Strands (from AWS) is a lean model-driven ReAct loop of three parts: a model (Bedrock native default, Anthropic API supported), tools written as functions with the @tool decorator that builds the spec from type hints, and a prompt. It consumes MCP servers and separates agent state (a key-value store outside the model context), a conversation manager (sliding-window history by default), and a session manager (persist and restore).
- ▶ PydanticAI (from the Pydantic team) is also model-driven but its differentiator is type-safe, validated output: you set output_type, Pydantic validates the result, and the returned value is statically typed, moving whole classes of errors from run-time to write-time. @agent.tool receives RunContext while @agent.tool_plain does not, a tool can raise ModelRetry, and Claude is reached via AnthropicModel.
- ▶ The clean split is graph-based (LangGraph alone, you author the flow) versus model-driven (Strands and PydanticAI, the model directs its own steps); within model-driven, needing typed validated output points to PydanticAI, otherwise Strands. Avoid cross-attribution: StateGraph, reducers, and checkpointers are LangGraph only, validated typed output is PydanticAI's signature not Strands's, and all three wire to Claude so framework choice never blocks it.

Domain 2: Applications and Integration

Exam weight: 33%

Section 2A — Understanding Requirements

Deriving Functional Requirements from Business Needs *(Lessons 19 and 20)*

Turning a stakeholder outcome into a concrete, buildable behavior that a developer can code and QA can verify.

- ▶ Move left to right through three anchor terms: a business need is the outcome someone wants (states value, not behavior), a functional requirement is a specific behavior phrased as input and expected output, and acceptance criteria are the concrete conditions that prove it is met. Each step is more testable than the last.
- ▶ Work outcome first: name the business need in plain language, break it into user-observable capabilities, then map each capability to exactly one Claude interaction. Keep capabilities observable and small enough to map to a single interaction, never internal plumbing.
- ▶ Business intent is how the product should feel (fast, trustworthy, effortless) and names no buildable behavior; functional behavior is what the app actually does with an input, like returning valid JSON with the caller's name, email, and plan. Translate the feeling into a concrete contract.
- ▶ The three interaction tiers climb in power, cost, and complexity: a single call (one request, one response: classification, summarization, extraction, direct questions), a workflow (a fixed code-controlled sequence decided at design time), and an agent (an open-ended loop where the model chooses each next action at run time). Always reach for the simplest tier that meets the need.
- ▶ An agent is never the default: commit only after passing four checks, complexity (truly multi-step and hard to script), value (justifies higher cost and latency), viability (Claude can do the open-ended task), and cost of error (a wrong step is recoverable, not catastrophic). Any weak check means drop to a simpler tier. Also watch for behaviors that are really infrastructure constraints (latency, region, throughput) in disguise, which get fixed in architecture, not the prompt.

Infrastructure Requirements and Solution-Architecture Fit *(Lessons 21 and 22)*

The four hard budgets, latency, throughput, region, and cost, that shape a Claude application's architecture before any feature does.

- ▶ A functional requirement states what the app must do (behavior from a business need); an infrastructure requirement states what the solution architecture must provide underneath. 'Respond in under two seconds' is not a feature, it is a latency budget.
- ▶ Set a latency budget as a number: take the worst-case response time a user will accept, subtract your own network and processing overhead, and what remains is the budget the model call must fit inside. A tight budget rules out multi-step agent loops and large contexts.
- ▶ Three delivery modes fit different budgets: synchronous (caller waits for the whole answer, simplest but full latency felt), streaming (tokens arrive incrementally so output starts almost immediately, often

how you meet a perceived-latency target), and batch (many requests submitted together and collected later, trading real-time speed for lower cost).

- ▶ Three levers hit a tight budget: prompt caching reuses a stable prefix so repeated reads bill at a fraction of base input price (cache writes cost a premium, so it only pays off when reused), message batches run deferrable work asynchronously at half price with results matched by identifier not position, and right-sizing matches the model tier to the task. When volume is high, caching and batching become requirements, not optional polish.
- ▶ Region and residency can decide where calls physically run: the first-party API is the simplest path but you inherit whichever regions that endpoint serves, while running Claude inside a major cloud platform can pin traffic to a specific region to satisfy data-residency rules.

Section 2B — Systems Life Cycle

Systems Life Cycle: Develop, Implement, Operate, Maintain *(Lessons 23 and 24)*

The four life-cycle phases every Claude application passes through, and what each phase demands of the builder.

- ▶ The four phases are develop (prototype prompts and integrations against the Messages API, proving the idea), implement (wire auth, secrets, error handling, and deployment for the target environment), operate (run it live watching latency, token usage, and error/refusal rates), and maintain (migrate models, version prompts and schemas, regression-test upgrades).
- ▶ Every request sets three essentials: an HTTP POST to the v1 messages endpoint sending and receiving JSON, the x-api-key header (kept in a secret manager, never hard-coded), and the required anthropic-version header pinning behavior to a dated API version.
- ▶ At the integration edge, triage errors by type: a 429 means rate-limited (back off and retry), 5xx codes are server-side (usually safe to retry), and a 401 is bad auth that retrying will never fix, so surface it immediately instead of hiding it in a retry loop.
- ▶ Commit shared config (CLAUDE.md and project settings.json) and pin an exact model ID so a teammate's run matches yours; keep settings.local.json gitignored. From 4.6 on, the dateless ID like claude-opus-4-8 is already the pinned snapshot, so there is no floating alias to avoid.
- ▶ A model ID is a fixed snapshot, not an evergreen pointer: Anthropic never rewrites the weights behind an existing ID, so an improved model always ships under a brand-new ID you opt into (though serving infrastructure can shift, causing small drift). Migration means regression-testing the new model against a frozen evaluation set and cutting over only when quality holds. Read failure signatures individually: 429 is rate limiting, a refusal stop reason (or a conversational decline under end_turn) is a policy issue not a retry, and max_tokens means the reply was cut off.

Section 2C — Claude API Mechanics

Messages API Structure and Data-Access Patterns *(Lessons 25 and 26)*

The fixed shape of every Messages API request and how to read the response object it returns.

- ▶ Three fields are required on every call: `model` (the Claude model as a string, picking the capability tier), `max_tokens` (the ceiling on reply tokens, where the response stops), and `messages` (the ordered array of conversation turns the model reasons over).
- ▶ Turns alternate by role (user or assistant); the very first turn must be a user turn or the call is rejected, and two same-role turns in a row silently merge into one. Content is either a plain string or a list of typed blocks (`text`, `image`, `tool_use`, `tool_result`) on the same endpoint.
- ▶ The API is stateless: it keeps no memory of past calls, so you resend the full conversation history every time and your code owns the transcript. Echo prior turns back as structured blocks, never flattened to a string, because flattening drops `tool_use` and thinking blocks (whose signature must pass back untouched) and breaks the next call.
- ▶ The system prompt is a top-level field separate from the `messages` array, not a user or assistant turn; it sets role, tone, and standing rules for the whole call and is passed as a string or a list of text blocks.
- ▶ Read a response by looping over content and branching on each block's type (a thinking or `tool_use` block can come first, so never grab index zero), branch on `stop_reason` to decide what to do next, and read usage for cost. `stop_details` is populated only on a refusal stop reason (Opus 4.7 and later) and is null otherwise, so check `stop_reason` first. To get full prompt size, add uncached `input_tokens` plus both cache read and write figures.

Tools, Streaming, and Vision *(Lessons 27 and 28)*

Defining tools Claude can call, steering that choice, and the mechanics of streaming responses and image input.

- ▶ A tool is three parts: a unique name (the label Claude uses to call it), a plain-language description (which Claude leans on heavily to choose well), and an `input_schema` (JSON Schema defining the argument shape). Set `strict` to true so generated input validates, which requires `additionalProperties` false and a required list.
- ▶ `tool_choice` has four modes: `auto` (default, Claude decides), `any` (forces at least one tool, Claude picks which), `tool` (forces one specific named tool), and `none` (blocks all tools, text only). Add `disable_parallel_tool_use` to hold Claude to a single call. Client tools (your own, plus `bash`, `text_editor`, `memory`, `computer_use`) run in your app and return `stop_reason` `tool_use`; server tools (`web_search`, `web_fetch`, `code_execution`, `tool_search`) run on Anthropic infra with results returned directly.
- ▶ Return every `tool_result` inside a single user message, each matched to its call by `tool_use_id`, with a result for every `tool_use` block Claude sent; skip one and the API rejects the whole turn. A server tool grouped in parallel with a client tool falls back to `stop_reason` `tool_use`.
- ▶ Streaming (`stream` set to true) delivers events in a fixed order: `message_start` opens with empty content (never read text from it), then `content_block_start`, a run of `content_block_delta` fragments you accumulate, and `content_block_stop` per block, then `message_delta` carries `stop_reason` and usage, then `message_stop`; ping and error can appear anywhere. A streamed `tool_use` arrives as `input_json_delta` `partial_json` fragments you concatenate and parse only at `content_block_stop` into a complete object. Streaming is unsupported inside the Batches API.
- ▶ Images pass as content blocks via three source types: `base64` (inline data with a `media_type`, works everywhere), `url` (fetched for you, small payload), or `file_id` (uploaded via the Files API, reusable across turns). Supported formats are JPEG, PNG, GIF, and WebP (animations read first frame only); place the

image before the text that refers to it, and label multiple images. Hard caps are 8000 by 8000 pixels and 10 megabytes base64; Claude reads images in 28 by 28 pixel patches at one visual token each, and newer models like Opus 4.8 handle a longer edge up to 2576 pixels. Vision understands images only: it never generates or edits, won't read metadata or name people, gives approximate counts and positions, and discards inline image content after the request.

Extended/Adaptive Thinking and Prompt Caching *(Lessons 29 and 30)*

The model-specific reasoning controls of adaptive thinking and effort, plus the byte-exact prefix mechanics of prompt caching.

- ▶ Adaptive thinking (thinking type adaptive, no budget_tokens) is the current path, and its default is model-specific: off on Opus 4.8 and Opus 4.7, on by default on Sonnet 5 (disable with thinking type disabled), and always on with no off switch on Fable 5.
- ▶ effort is nested at output_config.effort, not at the request root, with values low, medium, high (the default), xhigh, and max; xhigh is available on Fable 5, Opus 4.8, Opus 4.7, and Sonnet 5.
- ▶ Legacy budget_tokens returns a 400 on Fable 5, Sonnet 5, Opus 4.8, and Opus 4.7; it still works as a deprecated escape hatch on Opus 4.6 and is the only way to enable thinking on Sonnet 4.5 and Opus 4.5, so always name the model.
- ▶ Thinking blocks in the latest assistant message must be passed back exactly as returned or the next call returns a 400, and with thinking plus tools only tool_choice auto or none are allowed; thinking.display defaults to omitted on the newest models and setting it summarized changes visibility only, never billing.
- ▶ Prompt caching matches a byte-exact prefix rendered as tools, then system, then messages, marked with up to four cache_control breakpoints; reads cost 0.1x base input, 5-minute writes 1.25x, and 1-hour writes 2.0x, and a prefix must clear a per-model floor (1,024 tokens on Opus 4.8, 2,048 on Opus 4.7) that never carries across models.

Message Batches API: Realtime vs. Batch Tradeoffs *(Lessons 31 and 32)*

When bulk work should go asynchronous, and the custom_id discipline that keeps unordered batch results from silently misaligning.

- ▶ The Message Batches API runs asynchronously at 50% of standard price on both input and output tokens, trading away an immediate response and streaming for the discount; the synchronous Messages API is the only surface that streams.
- ▶ Two distinct windows govern a batch: the 24-hour processing window is the deadline to finish (unprocessed requests expire unbilled), while the 29-day retention window is how long finished results stay downloadable; most batches finish in under an hour, but that is best-effort, not a service level.
- ▶ Hard ceilings are 100,000 requests or 256 MB of total payload per batch, whichever hits first, and batches are not eligible for Zero Data Retention.
- ▶ Each entry pairs a unique custom_id (1 to 64 characters, only letters, numbers, hyphens, and underscores) with a standard params body; results return in any order, so you must match by custom_id and never by position, or data silently misaligns with no error.

- ▶ Batches keep tool use, vision, system messages, and thinking, so they are not a stripped-down mode; they reject stream, the speed flag, thread params (store and previous-thread), and cache-hint params, and max_tokens must be at least 1. Prefer the 1-hour cache TTL for shared prefixes, since concurrent out-of-order execution can expire a short one.

Invoking Claude via Third-Party Vendors (Bedrock, Vertex) *(Lessons 33 and 34)*

Reaching Claude through AWS Bedrock and Google Vertex AI with the same Messages API but different clients, credentials, model IDs, and feature gaps.

- ▶ Both vendors accept the identical Messages API request body and return the same streaming events, so prompt and parsing code stay unchanged; only the client, credentials, and model identifiers differ, and neither ever uses an Anthropic API key.
- ▶ On Bedrock the current path is bedrock-mantle (AnthropicBedrockMantle client, bare anthropic. model IDs like anthropic.claude-opus-4-8) for Opus 4.7 and later, while legacy InvokeModel or Converse (plain AnthropicBedrock client, profile-prefixed global./us./eu. IDs) is for Opus 4.6 and earlier; AWS credential precedence is constructor args, then environment variables, then the config file and standard chain.
- ▶ On Vertex two request pieces move: the model ID lives in the endpoint URL (each model is its own endpoint, ending in the rawPredict action) and anthropic_version is a body field equal to the literal vertex-2023-10-16; you authenticate with Google Cloud credentials and there is no x-api-key. Current models use the bare first-party ID (claude-opus-4-8), and dated snapshots use an @ separator (claude-opus-4-5@20251101).
- ▶ Shared feature gaps: no Files API or URL image input (images go inline as base64, capped at 5 MB each on Bedrock), no Message Batches API, no MCP connector, and no code execution or web fetch; prompt caching, extended thinking, client-side tool use, citations, and structured outputs all carry over.
- ▶ Region choice: global carries no premium and the widest model coverage, while multi-region (us or eu) and specific regions each add about a 10% premium. The key divergence is that web search runs on Vertex in its basic variant only, but does not run on Bedrock at all.

Section 2D — Software Engineering Foundations

REST APIs and JSON *(Lessons 35 and 36)*

The Claude API is stateless REST over HTTPS, and every call is one JSON object out and another JSON object back.

- ▶ The API is stateless, so the server keeps no memory between calls and you must resend the full conversation each time; almost everything is a single POST to the /v1/messages path over HTTPS, and you pick the model inside the body, not in the URL.
- ▶ POST is neither safe nor idempotent (unlike a safe GET that only reads, or an idempotent DELETE that lands the same state on repeats), so each messages call is a fresh generation and a blind retry after a timeout can produce a second, separate completion.
- ▶ Required headers are x-api-key (your secret API key), anthropic-version (pins dated API behavior), and content-type application/json; missing the key or the version fails the call before Claude ever runs.

- ▶ Status codes read by family: 2xx success, 4xx your fault, 5xx server fault (retry with backoff); specifically 401 is not authenticated, 403 is authenticated but not permitted, 429 is a rate limit (slow down and back off), and 400 is a malformed body that you must fix before retrying.
- ▶ The request body is one JSON object with model, the messages array, and max_tokens (plus optional system, temperature, tools); the response returns a content array of blocks, a stop_reason (end_turn, max_tokens, tool_use, or stop_sequence), and usage token counts. Message content is a list of typed blocks, not a plain string, and images ride in as base64, which inflates size by roughly a third.

Asynchronous Programming Patterns *(Lessons 37 and 38)*

Running many Claude calls at once without freezing, without flooding the rate limit, and without letting one failure sink the batch.

- ▶ Claude calls are I/O-bound, spending most of their time waiting on the network, so async and await let a coroutine issue a request and yield control to the event loop instead of freezing; await is the point where your code yields, not where it blocks, and it reclaims idle time without extra threads.
- ▶ Async delivers concurrency on a single thread, where tasks interleave and only one runs at any instant, which is not parallelism (simultaneous execution across multiple cores); keeping that distinction straight explains much of async performance behavior.
- ▶ Fan out independent calls with gather (start all requests in flight, then await them as a group) so total time trends toward the slowest call rather than the sum, but always bound concurrency with a semaphore that caps in-flight requests, since unbounded fan-out blows past the rate limit into a wave of 429 errors.
- ▶ Streaming yields an async iterator consumed with async for: text arrives as deltas you append in order until the content block completes, so the user sees output early. On a 429 (Too Many Requests), apply backpressure and retry with exponential backoff plus random jitter, and only retry transient errors (timeouts, 429, 5xx), never a permanent one like a 400.
- ▶ Isolate failures across branches by gathering outcomes so one exception does not cancel the rest, capturing each branch's result or exception, then handling or retrying only the failures after the group returns. Note that many parallel real-time calls (full price, bounded by your rate limit) is not the Batches API (one async job at half price).

Version Control and SDLC Integration *(Lessons 39 and 40)*

A Claude application is source code, so its prompts, schemas, and config belong under the same git workflow, review, and versioning as any codebase.

- ▶ Git rests on three building blocks: a commit is a content-hashed snapshot where the same hash always points to identical state, a branch is a movable pointer to a line of work, and a tag is a fixed label pinning one commit to a release you can rebuild.
- ▶ Rebase local work, merge shared work: merge records a merge commit that keeps history honest but tangled, while rebase replays commits into a clean straight line but rewrites history, so never rebase a branch others already have.
- ▶ For a Claude app, source is more than code: prompts, tool and output schemas, and config files that set behavior and permissions are all text and all belong in the repository, versioned beside the code.

- ▶ A one-line prompt edit can swing model behavior more than a hundred lines of code, so prompt and config changes earn the exact same branch-and-review flow as code, with the diff showing the precise before-and-after wording.
- ▶ Commit project CLAUDE.md and the project settings file so everyone who clones gets the same instructions and permissions; keep the local settings file and personal CLAUDE.local.md gitignored so individual tweaks never override a teammate's setup.

Code Review and Refactoring *(Lessons 41 and 42)*

Integration code that calls Claude gets reviewed like any other logic, and refactored on the same discipline: preserve behavior, lean on tests, and keep a human approving.

- ▶ Review integration code on three axes: error handling that classifies each failure by type first, response validation against a schema or business rules with re-prompting on failure, and defensive parsing that anticipates malformed or incomplete output.
- ▶ One question settles every retry loop: is it retrying only errors that can clear? Timeouts, rate limits, 5xx, and network blips are the only retryable errors; a bad request, invalid schema, or 4xx auth failure returns the same result every attempt, so a retry wrapped around one is a defect that burns budget.
- ▶ Validation must be the full loop of generate, validate, re-prompt, not a one-shot check; watch for validation that only checks syntax or only runs at the very end, since semantic checks plus a correction path are what catch a plausible but wrong result.
- ▶ Refactoring restructures working code without changing what it does, and it stays disciplined only when it is behavior-preserving, test-backed, and kept out of any commit that also changes a feature.
- ▶ The highest-value refactor is isolating every Claude call behind one client interface so model name, headers, and retries live in a single module; extracting duplicated prompt text into shared named fragments delivers the same win, making a model pin or prompt bump a one-place change.

Section 2E — Claude Application Design

Instruction Interpretation Across Interfaces *(Lessons 43 and 44)*

One Claude model reaches you through several surfaces, and it is the scaffolding around the model, not the model itself, that makes behavior differ.

- ▶ The three code surfaces load different amounts of scaffolding: the raw API auto-loads nothing and takes instructions only through the system parameter, the Agent SDK runs the agent loop with a minimal default prompt built for tool calling, and Claude Code loads a full system prompt, the whole CLAUDE.md hierarchy, and auto memory at startup.
- ▶ The SDK's default prompt is not the Claude Code prompt; to match the command-line agent you set the `claude_code` preset explicitly, and Claude Code uses its full prompt and config stack by default even in headless print mode.
- ▶ Setting sources, not the preset, control CLAUDE.md loading in the SDK: the project source loads the repo file and the user source loads the home-config file, both enabled by default, but passing an empty sources list loads no CLAUDE.md at all.
- ▶ Hosted chat surfaces are surface-scoped: profile and Project custom instructions in `claude.ai` (or Claude Desktop) apply only inside that web app and never reach Claude Code sessions or raw API calls.

- ▶ When behavior differs across surfaces, isolate the interface before blaming the model: reproduce the same prompt on a surface you control, check what config and tools each one actually loaded, and confirm the output; most surprises trace back to scaffolding.

Content Boundaries and Schema Design *(Lessons 45 and 46)*

Schemas are the contract between Claude and your code, and content boundaries are the line between instructions you trust and data you must not.

- ▶ Structured outputs and strict tool use are two jobs for the same engine: set `output_config.format` to a `json_schema` to constrain the whole final response, and set `strict` true on a tool to guarantee its `input_schema` when Claude calls a function.
- ▶ Under structured outputs you must set `additionalProperties` to false so the model cannot invent extra keys, and any value other than false is unsupported; enum restricts a field to fixed scalar values and `const` pins it to one value.
- ▶ The schema layer does not enforce minimum, maximum, `multipleOf`, `minLength`, or `maxLength` (the SDK strips length hints into the description) and does not support self-referencing schemas, so range-check numbers in your own code and flatten tree-shaped data.
- ▶ Guarantees and limits: a supported schema yields always-valid JSON with required-field enforcement and no retries, but a refusal or max-tokens cutoff may not match; up to twenty strict tools and twenty-four optional parameters per request, with grammar compilation cached for a day.
- ▶ Prompt injection and jailbreak need different defenses: injection smuggles instructions through untrusted input (user messages, retrieved documents, and tool results all count) and is defended by isolation, least privilege, and output validation, while a jailbreak defeats the model's own safety policy and is defended by its safety training.

Session Hygiene *(Lessons 47 and 48)*

A context window only ever accumulates, nothing leaves it on its own, so managing session state and knowing which reset to use is active engineering.

- ▶ The context window holds the system prompt, tool definitions, full history, and every tool result at once, and none of it clears automatically; large stale tool output is the biggest budget-killer since it is reprocessed every later turn until you prune it.
- ▶ A bloated window costs three ways: higher bills because every token is reprocessed each turn, added latency, and drift where low-signal history crowds out the context that matters and weakens instruction-following.
- ▶ Ephemeral context in the live window vanishes at truncation or session end, so anything that must persist belongs in durable state on disk; a new Claude Code session starts empty and is re-grounded by `CLAUDE.md` (re-injected even after compaction) and auto memory under a `MEMORY.md` index.
- ▶ The three resets run lightest to heaviest: compact summarizes older history while keeping recent turns and the thread, clear empties the window in the same session, and start fresh opens a new session re-grounded from config when the old one has drifted or switched tasks.
- ▶ Compaction is not lossless: project-root `CLAUDE.md` re-injects from disk and recent decisions survive in the summary, but early prompt detail and nested `CLAUDE.md` files (which reload only when Claude

next reads a file in that subdirectory) can fall away, so keep anything that must survive in project-root config, not buried in history.

Plugin Management *(Lessons 49 and 50)*

Plugins bundle new capabilities into one installable directory, and the marketplace-then-install flow plus a small command set control them.

- ▶ A plugin is a self-contained directory identified by a plugin.json manifest that declares what it provides; a single plugin can bundle slash commands, skills, subagents, hooks, MCP servers, LSP servers, and background monitors, all installing and updating as one unit.
- ▶ Installing is always two separate steps: register the marketplace (the catalog) first, which installs nothing on its own, then install the plugin by name scoped to that marketplace; the official marketplace ships already available, so you only add community ones.
- ▶ Disable pauses a plugin while keeping it installed for instant re-enable with no re-download; uninstall deletes the plugin and its files for good; running the plugin command with no arguments opens an interactive manager with four tabs: Discover, Installed, Marketplaces, and Errors.
- ▶ reload-plugins applies enable, disable, and install changes to the running session so you never restart Claude Code; at install time you also pick a scope deciding whether the plugin is just for you or the whole project.
- ▶ Three project layers do different jobs and must stay separate: plugins add capabilities (the commands and tools), CLAUDE.md gives natural-language instructions read as context, and settings.json sets permissions and other machine-readable options.

Section 2F — Configuration Management

CLAUDE.md Files and Hierarchy *(Lessons 51 and 52)*

CLAUDE.md is the developer-authored rule brief loaded every session, and multiple files across scopes layer by reading order rather than hard override.

- ▶ CLAUDE.md holds developer-authored rules loaded in full into context at session start; distinguish it from agent memory (durable state the agent writes, surviving compaction) and the context window (ephemeral, truncating working space).
- ▶ Claude treats CLAUDE.md as context to consider, not enforced configuration, so a rule can be missed when context is crowded; to truly block an action use a PreToolUse hook, which runs deterministically. Good content is project structure, conventions, and exact build/test/lint commands written verbatim.
- ▶ The whole file loads regardless of length and counts against the context window, so it is not free memory; aim to keep it under a couple hundred lines and prefer short, scoped directives over verbose prose. It arrives as a user or project message alongside the system prompt, not baked into it.
- ▶ Four scopes stack broad to specific: managed policy (machine-wide, loads first, no user can exclude it), user (home config, all your projects), project (repo root, committed, team-shared), and local (personal, gitignored, one project).
- ▶ Files are concatenated from the filesystem root down to your working directory, so the most specific file is read last and project tends to win over user; but this is composition not strict override, so if rules truly conflict Claude may pick either one. Subdirectory files load on demand when Claude reads a file

in that folder, and imports use an at-path reference resolved relative to the importing file, chaining to a maximum depth of four hops without saving any tokens.

settings.json Configuration *(Lessons 53 and 54)*

settings.json is the machine-readable control layer carrying permissions, hooks, environment, and model choice, and the same setting can be defined across four layers that resolve by precedence.

- ▶ Permission rules land on one of three decisions: allow runs the matching call with no prompt, deny blocks it outright so it never executes, and ask pauses to prompt you; these are enforced config, not suggestions the model can reinterpret.
- ▶ A hook wires a deterministic script to a lifecycle event, declared in settings.json (not prose) by naming the event and the command. Five common events are PreToolUse (before a call), PostToolUse (after a call), Notification, Stop, and SessionStart; PreToolUse is where guardrails belong because it can actually stop an action, while PostToolUse can only log or react after the fact.
- ▶ A PreToolUse hook sets permissionDecision to one of four values: allow (approve, no prompt), deny (block), ask (hand decision to you), or defer (fall back to your normal permission rules); an empty object leaves the call unchanged, behaving like allow.
- ▶ Four config layers stack broadest to most locked-down: user (~/.claude/settings.json, all your projects), project (.claude/settings.json, committed and team-shared), local (.claude/settings.local.json, auto-gitignored and personal), and managed (IT policy, machine-wide, cannot be overridden).
- ▶ Precedence highest to lowest is managed, then command-line arguments, then local, then project, then user; the committed project file needs workspace trust before its allow rules take effect, so secrets and machine-specific values belong in the local file. Most settings.json keys reload live, but model and output style need a restart.

Model Version Pinning and Prompt Versioning *(Lessons 55 and 56)*

A Claude model ID is a frozen snapshot you pin for reproducible production behavior, and prompts and config are versioned artifacts tested and rolled back like code.

- ▶ Each model ID names a fixed snapshot whose weights Anthropic never changes; an updated model always ships under a brand new ID, so the same ID returns the same model tomorrow. Dateless IDs (4.6 generation and later, like claude-opus-4-8) are themselves the pin, not a moving latest pointer; older models use a trailing year-month-day stamp.
- ▶ Aliases, a pre-4.6 convenience only, point at the newest dated snapshot and trade reproducibility for freshness; they are not offered for newer dateless IDs. Pinning locks the weights but not routing, safety classifiers, or sampling, which sit outside the weights and can still shift slightly, and every ID carries its own deprecation schedule making migration ongoing maintenance work.
- ▶ Treat prompts as versioned artifacts in source control, reviewed like a pull request and tied to the exact model ID they were tuned for, since a prompt and model behave as a pair; version the prompt text, the pinned model ID, and the settings.json/permissions/hooks config together as one unit.
- ▶ The anthropic-version header pins request and response behavior to a dated API version and is required (SDKs set it automatically); within a version Anthropic only adds fields and never breaks existing ones, so treat moving to a newer version as a deliberate, tested upgrade.

- ▶ Never promote a new model or prompt version straight to production: run the candidate against a frozen regression eval set and promote only when it matches or beats the current version's results. Versions get tracked three ways: source-control commits, a plugin's SemVer version field, and the git commit SHA as fallback when a plugin omits its version. Roll back by repointing the pin to the last known-good ID, restoring the matching prompt, and confirming against the same evals.

Plugin Dependencies *(Lessons 57 and 58)*

Plugins declare the other plugins they need, and install, enable, disable, and cleanup all cascade across that dependency graph, resolved reproducibly through committed declarations and git tags.

- ▶ A plugin declares needs in a dependencies array in its manifest, where each entry is either a bare plugin name (tracks whatever the marketplace currently offers, so the version can shift) or an object adding a semver version range and optionally a different marketplace to source from; installing resolves the array and pulls in every dependency automatically, printing each addition.
- ▶ Semver ranges carry exact meaning: tilde (like ~2.1.0) allows patch updates only, caret (like ^2.0) allows minor and patch below the next major, and a comparator (like >=1.4) is loosest with the least protection; resolution always fetches the highest tagged release satisfying the range.
- ▶ Enabling a plugin transitively enables its dependencies at the same scope, even ones whose own manifest sets defaultEnabled to false; disabling is blocked while another enabled plugin still depends on it, and Claude Code prints a chained command to disable the dependents first.
- ▶ When several plugins constrain the same dependency, Claude Code intersects their ranges and picks the highest version satisfying all; if they cannot be reconciled the install fails with a range-conflict error and the existing setup is preserved. Removing a plugin leaves auto-installed dependencies as orphans, and plugin prune sweeps only those auto-pulled extras, never plugins you installed directly.
- ▶ Version ranges resolve against repo tags in the plugin-name--vX.Y.Z pattern (like logger--v2.1.0) that the author publishes with plugin tag; commit the manifest's dependencies array and the project settings recording enabled plugins so a teammate's clone resolves the same set. Cross-marketplace dependencies are blocked until the root marketplace allowlists the target, and four error codes read from plugin list --json pinpoint failures: dependency-unsatisfied (missing or disabled), dependency-version-unsatisfied (out of range), no-matching-tag (no published tag), and range-conflict.

Domain 3: Claude Code

Exam weight: 3%

Section 3A — Claude Code Operation

Core Components: Rules, Skills, Commands, Agents, Agent Memory (Lessons 59 and 60)

The three ways to steer Claude Code, why rules and skills load in opposite directions, and how subagents and agent memory extend work beyond a single context.

- ▶ Rules, skills, and commands load differently: rules are always-on instructions in CLAUDE.md loaded at session start and applied every turn, while a skill keeps only its description in context and loads its body only when the task matches. Rules persuade rather than enforce, so a hard block is a job for permissions or hooks.
- ▶ A skill is a folder with a SKILL.md entry file whose YAML frontmatter carries the description Claude matches on; the folder name becomes the slash keyword. Frontmatter fields are technically optional (name defaults to the folder, description to the first paragraph), but a bare file with a weak or missing description matches poorly and fires unreliably.
- ▶ Slash commands and skills were merged into one system. A file at .claude/commands/name.md and a skill folder at .claude/skills/name/SKILL.md both create /name and run the same way, so a custom command is a skill; when a skill and command share a name, the skill wins.
- ▶ A subagent is a separate Claude Code instance that runs a delegated task in a fresh, isolated context. It does not see your chat history or files already read, runs its own system prompt, tools, and permissions, and returns only a summary. Isolation is the reason to use one, not a limitation.
- ▶ Define a subagent as a Markdown file with YAML frontmatter whose body is its system prompt; only name and description are required, and omitting the tools field inherits the full set. Agent memory is distinct from CLAUDE.md: CLAUDE.md is instructions you author, memory is durable state the agent writes itself that survives compaction, split into on-by-default per-project auto memory and frontmatter-enabled subagent memory.

Session Management and Slash Commands (Lesson 61)

Every conversation in Claude Code is a session saved to disk, and the slash-command menu is how you reopen work, keep context lean, and run built-in or custom actions.

- ▶ A session is a single conversation tied to the project directory it was launched in and written to disk continuously, so closing the terminal or a crash loses nothing. Old sessions clear after a configurable retention window (30 days by default), and lookups are scoped to the current project directory and its worktrees.
- ▶ Three ways reopen past work: the continue flag reopens the most recent session in the current directory, the resume flag opens a picker or jumps straight to a named session, and resuming from inside a live session switches to a different past conversation without quitting.

- ▶ Compact and clear are different: compaction replaces older turns with a summary when the window fills, keeping recent decisions and the current task alive, while clear empties the window and starts fresh. Clear does not delete the conversation, the prior history stays saved on disk and remains resumable.

Headless / Streaming / Auto Modes, CLAUDE.md Hierarchy, Repo Init, settings.json (Lesson 62)

Taking Claude Code from interactive use into scripts and pipelines using the print flag, streaming output, approval modes, and the CLAUDE.md and settings.json layers that hold instructions and enforcement.

- ▶ Headless mode runs Claude Code non-interactively with the -p (--print) flag: it takes a prompt, runs to completion, prints to stdout, reads stdin so data can be piped in, and exits. Without -p, Claude Code opens an interactive session and waits, so a pipeline or cron job hangs forever waiting on input that never comes.
- ▶ Headless output has three formats: plain text (the default, one block at the end), structured JSON (result plus session id, token usage, and total cost for scripts to parse), and stream-json (newline-delimited events emitted as work happens). Streaming input is the default in the Agent SDK and lets a caller show progress live.
- ▶ Claude Code documents six permission modes on a spectrum from full oversight to none; four matter: Default/Manual asks before every edit and tool call, accept-edits applies file edits and common filesystem commands without prompting but still asks for other shell commands, plan mode proposes actions and waits, and dontAsk is the documented mode for unattended or CI runs where only pre-approved tools run and the session never waits.

Domain 4: Eval, Testing, and Debugging

Exam weight: 3%

Section 4A — Debugging and Error Handling

Error Type Identification and Recovery Strategy Selection *(Lessons 63 and 64)*

Classifying a failure by whether it clears on retry and where it originated is a distinct step that must come before you choose any recovery.

- ▶ Transient errors (timeouts, rate limits, network blips, 5xx) mean the request was well-formed but the system was briefly unavailable, so the same call can succeed on retry; permanent errors (400, 401, 403, 404, 413) sit in the request itself, so repeating it unchanged just reproduces the failure. The tell: could waiting and trying again plausibly help?
- ▶ Know the status codes: transient signals are 429 `rate_limit_error`, 500 `api_error`, 504 `timeout_error`, and 529 `overloaded_error`; permanent signals are 400 `invalid_request_error`, 401 `authentication_error`, and 413 `request_too_large` (Messages API caps a single request at 32 megabytes).
- ▶ The error body has three fields: branch your code logic on type (a stable machine-readable label), log message (human-readable, never parse it since wording can change), and always capture `request_id` (echoed in the response header) for support tickets. Origin splits into model-output errors (wrong plan, bad tool argument, truncated answer riding inside a 200) versus integration errors (request, parsing, auth, endpoint) that surface as an HTTP status; a streamed error can arrive after a 200, hiding a transport failure behind a success code.
- ▶ Recovery follows the type, never the surface symptom: retry with exponential backoff for transient only (honor the `retry-after` header on a 429, cap attempts so it fails loudly); change the payload, credentials, or size for permanent 4xx (a 409 conflict is resolve-first, then retry); adjust the prompt, raise a limit, or fall back for model-output problems.
- ▶ Model-output failures on a 200 have four fixes: `max_tokens` truncation means raise the cap or continue; `model_context_window_exceeded` means prune history or compact; a refusal means read `stop_details` then adjust the prompt or switch models; a clean-but-wrong answer is a prompt-and-eval problem, not a network retry. Silent failures raise no error code and are caught only by validating output; escalate to a human carrying the `request_id` and error type when no automated recovery fits.

Trace Analysis and Problem-Origin Isolation *(Lesson 65)*

The trace, the ordered record of prompts, tool calls, results, thinking, and errors, is your primary evidence for isolating whether the integration layer or the model output diverged before you fix anything.

- ▶ Isolate the origin to one layer before editing: integration-layer failures wrap the model (API request, endpoint, auth, response parsing, tool harness) and surface as HTTP 4xx/5xx or transport errors in the event stream; model-output failures ride on a successful 200 and appear in the content or `stop_reason`.
- ▶ Read the trace outward-in, one checkpoint at a time: check HTTP status first (4xx/5xx means integration); on a 200 read the stream for a late error event; on a clean 200 inspect `stop_reason` and

content; a thrown or empty tool result points back at your harness. Stop at the first checkpoint that explains the divergence. Streaming returns the status code up front before generation finishes, so a mid-stream failure surfaces only as a streamed error event, still an integration failure despite the 200.

- ▶ On a clean 200 the `stop_reason` is the model-output tell: `max_tokens` is truncated-but-valid (raise the cap or continue), `model_context_window_exceeded` filled the model's total context window not your requested cap (prune history or compact), and refusal means Claude declined on policy grounds with `stop_details` carrying the category.

Domain 5: Model Selection and Optimization

Exam weight: 17%

Section 5A — LLM Fundamentals

LLM Basics: Tokens, Context Windows, Sampling, Non-Determinism, Next-Token Generation *(Lessons 66 to 68)*

Tokens are the shared unit for both context limits and billing, sampling picks each one probabilistically, and generation runs strictly left to right.

- ▶ A token is a sub-word chunk averaging about four characters or 0.55 words, so word counts are only a loose proxy; the `count_tokens` endpoint returns the exact count for the specific model you pass it.
- ▶ Tokenizers are model-specific: Opus 4.7 and later, plus Fable 5 and Sonnet 5, split text more finely and can produce roughly 30% more tokens for the same input, so re-baseline estimates whenever you switch model.
- ▶ The context window is one shared pool covering system prompt, tool definitions, full history, tool results, and current generation; Fable 5, Opus 4.8, and Sonnet 5 attend to 1M tokens (about 555,000 words) while Haiku 4.5 attends to 200,000, and output tokens (including thinking) bill higher than input.
- ▶ Overflowing the window returns an error, not an automatic summary, so you prune or compact history to stay under budget; a full 1M-token request bills at the standard rate with no long-context surcharge.
- ▶ Sampling draws each token from a probability distribution, so identical prompts can return different outputs and temperature, `top_p`, and `top_k` now return an HTTP 400 (temperature zero was never fully deterministic); generation is autoregressive and left to right with no lookahead, so anchor format early and validate every response defensively rather than expecting a byte-identical string.

Model Options: Fast Mode, Thinking, Effort *(Lessons 69 and 70)*

Fast mode, thinking, and effort are three separate levers you tune on a single call, and each one's availability depends on the exact model.

- ▶ Fast mode is a latency lever that pushes the same model up to 2.5x higher output tokens per second at premium pricing; it ships as a research preview behind a beta flag with a top-level speed option, runs only on Opus 4.8 (the durable home) and Opus 4.7 (deprecated), and is unavailable on the Batch API and third-party platforms.
- ▶ Fast mode buys speed on the same model without changing capability, unlike switching to a lighter, cheaper model such as Haiku 4.5, which changes capability and price at once.
- ▶ Adaptive thinking is the current way to let the model reason before answering, with no budget to tune; the legacy budget-tokens style survives only on older models like Sonnet 4.5 and Opus 4.5 and is rejected with a 400 on frontier models.

- ▶ Thinking defaults differ by model: Fable 5 always thinks with no off switch, Sonnet 5 thinks unless you disable it, Opus 4.8 and 4.7 are off by default until you enable adaptive thinking explicitly, and Haiku 4.5 has no adaptive mode and uses budget-tokens; reasoning stays hidden by default unless you surface it.
- ▶ Effort dials how much work the model spends across five levels (low, medium, high, xhigh, max) with high the default sitting in the upper band, not the middle; it is nested under `output_config.effort`, not a top-level field, and xhigh and max are limited to certain models, so tune effort before switching to a heavier model.

Fundamental Prompting: Zero-, Single-, Multi-Shot *(Lessons 71 and 72)*

Zero-, one-, and multi-shot prompting are a deliberate choice about examples, not a strength ladder, and the quality of examples matters more than their count.

- ▶ Zero-shot states the task with no worked examples, leaning on the model's pretrained skill and your wording; it is the right default for common, well-understood work like summaries, rewrites, translations, and simple classification, and keeps the prompt lean on tokens.
- ▶ A shot is a demonstration, a sample input paired with the exact output you want, placed in the message history as prior turns; an instruction such as a rule like 'keep it brief' is not a shot because it shows no worked output.
- ▶ One-shot supplies exactly one worked example to lock down an output format, a distinctive tone, or a niche shape that words describe poorly; add a second example only when it improves the result.
- ▶ Multi-shot uses two or more examples so the model generalizes a pattern rather than a single case; good examples are relevant (mirror the real use case), diverse (span edge cases to avoid overfitting the happy path), and clearly delimited from your instructions.
- ▶ Quality beats quantity: a few diverse examples outperform a long list of near-duplicates, every example costs input tokens on every call, and returns diminish once coverage is complete; on current models assistant prefills are rejected, so use examples or structured output for a fixed format.

Section 5B — Technical Fundamentals

SDKs That Wrap REST APIs *(Lessons 73 and 74)*

Every Claude call in any language is one HTTP request to a stateless JSON REST API, and the SDKs are just convenience clients over that same endpoint.

- ▶ The Claude API is a JSON-over-HTTP REST service where nearly everything routes through one endpoint, a POST to `/v1/messages`; tools, structured outputs, thinking, and prompt caching are all just fields or options on that same call, and supporting endpoints follow the same style (`/v1/messages/batches` for async at half price, `/v1/files`, `/v1/models`, and `/v1/messages/count_tokens`).
- ▶ A request names a model, carries a role-tagged messages array, and sets generation params like `max_tokens` and `stop_sequences`; the response returns a list of typed content blocks plus metadata such as stop reason and token usage.
- ▶ The API is stateless with no server-side memory or session, so you resend the full conversation history every turn and append each reply back into messages yourself, which is why context management matters so much.

- ▶ The SDKs are generated from one OpenAPI spec and send the identical POST /v1/messages under the hood, so JSON fields map one-to-one to SDK parameters (only naming-style differences) and new features land across all language clients together; the SDK handles auth (x-api-key and anthropic-version headers), automatic retries of transient errors like 429 and 5xx with exponential backoff (twice by default), typed request and response objects, and parsing the server-sent event stream.
- ▶ The SDK adds convenience, never capability: it exposes only what the REST endpoint supports, so unsupported in the API means unsupported in the SDK, and raw REST and the SDK are interchangeable since both hit the same endpoint; the default request timeout is around ten minutes, so stream very long generations rather than blocking.

Websockets and Connection Fundamentals *(Lessons 75 and 76)*

Claude's real-time streaming is one-way server-sent events over an ordinary HTTPS connection, never a websocket.

- ▶ Setting stream to true keeps one HTTP response open and writes labeled server-sent events with small JSON payloads as tokens generate; the endpoint and request body are otherwise unchanged, and streaming beats a buffered response for perceived speed and for long generations that a single blocking call would time out on.
- ▶ Stream events follow one guaranteed order: message start (a message with empty content, never the text itself), content block deltas (the workhorse text and tool fragments), then message stop (response complete); you accumulate deltas into a buffer to rebuild the full message.
- ▶ Tool arguments arrive as input_json_delta events carrying partial JSON that will not parse mid-stream, so parse only once the content block's stop event lands, when the tool input is guaranteed to be a complete, valid object.
- ▶ A websocket is a persistent full-duplex (two-way) connection built for constant back-and-forth like live chat or multiplayer games; Claude never uses one, every call streaming or not is a plain HTTPS request, so any design assuming a websocket to Claude rests on a false premise.
- ▶ Plan for a long-lived stream to drop and leave a partial response: retry transient errors (timeouts, 429 rate limits, network blips, 5xx) with exponential backoff, but fix permanent ones (bad requests, invalid schemas, auth failures) instead of retrying, ignore keep-alive ping events, handle mid-stream error events, and skip unknown event types.

Section 5C — Model Selection and Tradeoffs

Opus vs. Sonnet vs. Haiku: Use Cases and Quality/Latency/Cost Tradeoffs *(Lessons 77 to 79)*

Claude's tiers span top capability to top speed, and because capability, latency, and cost climb together, the lightest tier that clears the quality bar is usually the right pick.

- ▶ Opus (Opus 4.8, Anthropic's recommended default) is the most capable everyday tier for hard reasoning, multi-step coding, large refactors, and long-running agents, at the highest per-token cost and higher latency; Fable 5 sits above it for the very hardest, long-running work.
- ▶ Sonnet 5 is the balanced tier and the best combination of speed and intelligence, sized for high-volume production work at much lower cost and latency than Opus; Haiku 4.5 is the fastest, cheapest tier with near-frontier smarts, best for classification, routing, extraction, and lightweight sub-agent steps.

- ▶ Context windows split the lineup: Fable 5, Opus 4.8, and Sonnet 5 each attend to 1M tokens while Haiku 4.5 attends to 200,000, so a context-heavy prompt can sail through on Opus yet overflow Haiku.
- ▶ Latency and cost track capability: Haiku is fastest and cheapest, then Sonnet, then Opus, with Fable slowest and priciest, and output tokens bill higher than input on every tier.
- ▶ Match tier to workload by reading task difficulty, latency budget, and cost ceiling; either start light (Haiku) and upgrade only at a real capability gap, or start capable (Opus) and optimize down, and mix tiers in one pipeline (for example Haiku tags tickets, Sonnet drafts replies, Opus held for escalations) instead of sizing everything to the hardest step, avoiding both over-provisioning and under-serving.

Breaking Behavior Changes Across Model Releases *(Lessons 80 and 81)*

Migrating to a newer Claude model can turn a working call into a 400 error, because parameter support is model-specific and older parameters get retired in favor of cleaner mechanisms.

- ▶ A breaking change is when a newer model rejects a parameter an older one accepted, returning an HTTP 400 (a bad request on your input, not a server outage) with your code unchanged; the three classic casualties on Fable 5, Opus 4.8, Opus 4.7, and Sonnet 5 are sampling params (temperature, top_p, top_k), final-turn prefill, and the legacy thinking budget.
- ▶ Each retired parameter has a clear swap: replace sampling with prompting plus the effort level, replace the budget-tokens thinking mode with adaptive thinking, and replace an assistant prefill with a structured output format or a system-prompt instruction.
- ▶ Thinking behavior is model-specific and does not port forward: Opus 4.8 and Sonnet 5 use adaptive thinking (the old budget style returns a 400), Haiku 4.5 still uses budget-tokens with no adaptive mode, and Fable 5 keeps thinking always on so disabling it is rejected.
- ▶ Pin an exact model ID like `claude-opus-4-8` rather than a floating alias; from the 4.6 generation on, IDs are dateless fixed snapshots whose weights never change, so behavior shifts only when you change the ID, whereas a pre-4.6 floating alias can shift behavior with no code change.
- ▶ Treat migration as recurring maintenance driven by deprecation: a deprecated model gets a retirement date and later returns a 404 (Opus 4.1 is deprecated; Sonnet 3.7 and Haiku 3.5 have retired), so test against a frozen eval set on the exact target model before cutting over, re-baseline token counts (the newer tokenizer counts about 30% more), and keep monitoring because serving-side changes can shift behavior even on a pinned ID.

Section 5D — Cost and Token Management

Token Usage Tracking and Cost Modeling *(Lessons 82 and 83)*

Every response carries a usage object with exact token counts, and reading it correctly is the raw material for a cost model you can trust before you ship.

- ▶ The usage object rides back on every response reporting token counts for that one call only, not a running total, so summing across calls into your own ledger is your job.
- ▶ Input tokens (system prompt, history, tool definitions, current message) bill at the base rate, while output tokens (the visible answer plus thinking) bill several times higher on every current model, so a short prompt with a long reply can cost more than its input suggests.

- ▶ When caching is on, `input_tokens` is only the uncached remainder, not the full prompt, so true prompt size equals `input_tokens` plus `cache_creation_input_tokens` plus `cache_read_input_tokens`; use the `count_tokens` endpoint (model-specific, no generation charge) to size input before sending, passing the same model, messages, system prompt, and tools.
- ▶ Per-call cost is input tokens times the input rate, plus output tokens times the higher output rate, plus cache and any server-tool charges, with each cache field weighted by its own rate (reads deeply discounted, writes at a premium), so pricing cached tokens at full input rate overstates cost.
- ▶ Build a workload model by estimating tokens per request, multiplying by expected call volume, and finding the dominant driver; then cut spend with four levers: run the lightest adequate tier, trim prompts and history, cache a stable prefix so repeats bill at the cheap read rate, and send non-urgent work through the batch path for a large discount.

Caching for Cost: Prompt Caching and Cache Check-Pointing *(Lessons 84 and 85)*

Prompt caching trades a one-time write premium for cheap repeat reads, and a hit depends on a byte-exact, correctly ordered prefix that clears a per-model size floor.

- ▶ Cache reads bill at 0.1x base input, a 5-minute (default ephemeral) write at 1.25x, and a 1-hour write at 2.0x; a 5-minute entry pays off after roughly one reuse while a 1-hour entry needs two or more reads, and each read resets the entry's lifetime so continuous traffic keeps a 5-minute prefix warm indefinitely.
- ▶ Cache only big, stable, frequently-hit prefixes such as a large system prompt, a fixed tool set, or a long reference document; a prefix hit only once, content that changes every call, or a prefix under the model's token floor loses money, so plain resending is cheaper when reuse is rare.
- ▶ The request renders in one fixed order, tools then system then messages, and the cache key is the exact rendered bytes matched byte-for-byte and prefix-only; place up to four `cache_control` breakpoints at stable boundaries and push volatile per-request data after the final breakpoint.
- ▶ Caches invalidate silently with no error from a timestamp, UUID, or session ID baked into the system prompt, from serializing tools or data with unstable key order, from changing the tool set (tools sit at the front of the prefix), or from switching models (caches are model-scoped).
- ▶ A prefix must clear a per-model minimum to cache at all, 1,024 tokens on Opus 4.8 and 512 on Fable 5, so the same prefix can cache on one model and silently refuse on another (returning `cache_creation` of zero with no error); verify hits by watching `cache_read_input_tokens` climb on repeated identical calls, and note the Batch API 50% discount and data-residency multipliers stack on cache pricing.

Domain 6: Prompt and Context Engineering

Exam weight: 11%

Section 6A — Context Engineering

Context Window Management *(Lessons 86 and 87)*

The context window is a finite token budget you own, and managing it means keeping content informative yet tight rather than filling and forgetting.

- ▶ The context window holds everything the model attends to in one request: system prompt, tool definitions, conversation history, tool results, and the current generation, and raw tool results are usually the single biggest source.
- ▶ Accumulation is additive; nothing is dropped automatically and the API never auto-summarizes on overflow, so pruning and compaction are the application's job. Drift, bloat, and overflow all trace to an unmanaged window, and drift is fixed by re-grounding and pruning, not a louder prompt.
- ▶ Prune verbose tool output first, it is the biggest single win. One file read or API response can dump tens of thousands of tokens, so cut each result to its load-bearing fields before it lands in history.
- ▶ Tool-result clearing is a mechanical edit with no inference cost: it swaps old results for placeholders, keeps the three most recent tool-use and result pairs, and lets you exclude specific tools, but it invalidates the prompt cache from the edit point.
- ▶ Compaction runs a model to summarize older turns, so unlike clearing it is not free and is lossy; its default trigger is 150K input tokens with a 50K floor, and high-level facts and decisions survive while verbatim phrasing is lost, so save anything needed word for word.

Context Isolation via Subagents and Multi-Step Workflows *(Lessons 88 and 89)*

How subagents and staged workflows keep verbose discovery out of the parent's context so every step runs against a lean, focused window.

- ▶ Context isolation delegates a scoped subtask to a subagent that returns only its distilled result, typically 1,000 to 2,000 tokens, keeping the subtask's intermediate tokens out of the parent's window. The costliest tokens in a run are usually discovery, not the final answer.
- ▶ A subagent starts fresh and does not inherit the parent's conversation; anything it needs must go into the delegated prompt explicitly, and assuming shared context is the single most common handoff bug.
- ▶ The returned summary is the contract, not a window into how the subagent got there. Validate it at the parent before acting, and never reach back into the subagent's working trace, which is gone by design.
- ▶ Delegate when a subtask makes high throwaway noise, runs independently, or needs its blast radius contained; keep a step inline when its details are referenced downstream or it is a quick single lookup, since delegation carries real overhead.
- ▶ Prompt chaining breaks a task into sequential model calls where each step's output feeds the next, and a staged handoff forwards only the distilled output, not the full working trace. Chain explicitly to

inspect intermediate output, enforce a repeatable pipeline, or run a draft, review, refine self-correction loop.

Section 6B — Prompt Engineering

Instruction Clarity, System vs. User Placement, Output Constraints *(Lessons 90 and 91)*

Clear instructions plus correct content placement turn a loose prompt into a dependable one.

- ▶ A clear instruction is specific, unambiguous, and ordered; if a colleague with little context would misread it, so will Claude. Treat Claude like a brilliant new hire who lacks your norms, jargon, and house style.
- ▶ Tell Claude what to do, not what to avoid, positive framing steers far better than a wall of prohibitions, and adding the reason behind a rule lets Claude generalize the intent to cases you never spelled out.
- ▶ When output is inconsistent, the ask is almost always at fault, not the model, so tighten the wording and constraints before touching sampling, and add examples only after the instruction itself is clear.
- ▶ Stable role, behavior, and constraints go in the system prompt where they apply every turn; per-request data goes in the user turn; third-party or fetched content goes in tool results where Claude treats it as untrusted data. Keep your own instructions in a user turn, never in a tool result.
- ▶ Placement feeds prompt caching: a stable system prefix can be cached and reused across calls while volatile user content cannot, so mixing per-request data into the system prompt breaks the cacheable prefix. Output constraints of format, value, and length shape output and cut post-processing but stay best effort, and hard guarantees require structured outputs.

Few-Shot Examples and Iterative Refinement *(Lessons 92 and 93)*

How to choose and design few-shot examples, and how to refine prompts against a fixed eval set instead of a single lucky output.

- ▶ Zero-shot, one-shot, and multi-shot are a deliberate choice matched to task complexity and format specificity, not a fallback ladder; a clear zero-shot can equal few-shot on common tasks, and examples that don't earn their tokens are just bloat.
- ▶ Quality and coverage beat raw count, a small set of well-chosen examples spanning the real input range steers more reliably than a pile of near-duplicates that overfits the happy path. Strong examples are relevant, diverse, and clearly structured.
- ▶ Deliberately cover boundary and adversarial inputs, not just clean ones, since happy-path-only sets teach the model to assume every input is tidy. Wrap each example in its own labeled tags so Claude reads it as a demonstration rather than a directive, with three to five examples as the sweet spot.
- ▶ Refine in a loop: run the prompt against a fixed set, observe failures against clear criteria, then adjust and re-run the same set. One lucky output proves nothing, whereas a frozen eval set makes each change measurable, comparable, and safe to keep or roll back.
- ▶ The eval set needs three input types, representative, boundary, and adversarial. Treat prompts as versioned artifacts: keep them in source control, pin the model ID, and regression-test against the frozen set on every change; a one-line self-check that verifies the answer against criteria catches errors cheaply, especially in code and math.

Input Sanitization *(Lessons 94 and 95)*

How to treat untrusted input as data rather than instructions, and why sanitization must sit inside a stack of defenses.

- ▶ Untrusted input is any content the application did not author, user messages, retrieved documents, and tool results, and all of it should be treated as adversarial. Sanitization cleans, validates, and delimits it before the prompt so the model reads data as data.
- ▶ The most reliable delimiter is a labeled tag around the untrusted block with your instructions kept outside it; deliver third-party content in `tool_result` blocks, where the model is trained to treat instructions with skepticism, never in the system prompt where it inherits authority.
- ▶ Validate structured fields by type, range, and allowed values in developer-side code before the model call, rejecting or escaping anything that fails. Encoding untrusted content as a JSON string escapes quotes and tags, giving unambiguous delimiters an attacker cannot close early.
- ▶ Sanitization reduces injection risk but does not cure it, since a determined attacker keeps probing for a phrasing your filter never anticipated, so it must sit inside defense in depth. Prompt injection, where untrusted data plants instructions that hijack your application, is distinct from a jailbreak, where a user coaxes the model past its own safety policy.
- ▶ Three layers contain an injection: input isolation (sanitize and delimit), least privilege (scope tools and access narrowly and remove unused capabilities so a hijack reaches little of value), and output checks (screen what the model produces and treat tool and retrieval results as untrusted too). Injection is an input problem solved by architecture, not more safety training.

Section 6C — Output Handling

Structured Output Patterns and Response Validation *(Lessons 96 and 97)*

The reliability ladder for structured output, and the validate-and-re-prompt loop that catches a schema-valid but wrong answer.

- ▶ Structured-output reliability is a ladder: format instructions like a prose 'return only JSON' and few-shot examples shape output but guarantee nothing, while structured outputs and strict tool use rely on constrained decoding to guarantee schema conformance. Syntactically valid JSON is not schema compliance, a response can parse yet miss a required field, carry the wrong types, or invent an enum value.
- ▶ Two mechanisms enforce a schema and can be used alone or together: JSON outputs, the whole-response form set via a `json_schema` in `output_config.format`, and strict tool use, which sets `strict` to `true` so a tool's input schema is grammar-enforced. Constrained decoding removes parse errors, guarantees type-safe required fields, and eliminates schema-fix retry loops.
- ▶ Design the schema with precise names and pinned types, honest required flags with `additionalProperties` set to `false` on objects, enum constraints on categorical fields, and a description per field. On Claude 4.6 and later, prefilling the final turn to force JSON returns a 400 error, and structured outputs are the supported replacement.
- ▶ Guaranteed structure is not guaranteed correctness, so treat generation as a loop: generate, validate against the schema and your business rules, and on failure re-prompt with the exact error and the offending output. A syntax check proves the response parsed; a semantic check proves the values obey your rules and are complete.

- ▶ Structured output still breaks on edge cases: a refusal sets `stop_reason` to refusal with no schema-valid object, and a token-limit hit sets `max_tokens` with a truncated object, so branch on `stop_reason` before parsing and only trust `end_turn`. Compare enum values case-insensitively, expect required properties before optional ones regardless of declared order, and cap retries so a stubborn failure escalates instead of looping.

Defensive Parsing and Skepticism Toward Confident Output *(Lessons 98 and 99)*

How to parse model output tolerantly so one bad generation fails safely, and why a confident tone is never evidence a claim is correct.

- ▶ Defensive parsing rests on three habits: tolerant extraction that reads fields without assuming a well-formed payload, explicit failure handling with a defined path and never an unhandled crash, and no blind trust that treats every field as possibly absent or mistyped. Valid JSON is not valid data, so confirm each field exists and check its type before use, and guard accesses so a bad payload fails at the check, not three functions deep.
- ▶ Check `stop_reason` before parsing, since a `max_tokens` stop means a truncated object and a refusal means no schema-valid object. In the ticket-triage drill, a long ticket blew the token budget, generation stopped at `max_tokens` leaving unclosed JSON, and code that parsed before checking why the model stopped turned a known edge case into a mystery `KeyError`.
- ▶ Three safe responses to bad output are fail loud (raise a clear error at the boundary), fall back (a safe value, cached result, or degraded mode), and re-prompt (return the response with the error described). Silent failures return success with wrong content and parse cleanly, so only a semantic gate aimed at meaning can catch them.
- ▶ Confidence is a style, not a correctness signal, a model states a wrong answer with the same fluency as a right one and there is no built-in hedge on a mistake; detail and specificity can even make a wrong answer look more credible. The model predicts plausible text, not verified fact, so correctness must be established outside the model.
- ▶ Ground high-stakes output before trusting it using three verification sources: grounding that ties a claim to provided context, citation that names a checkable source, and validation that programmatically tests output against a schema or business rule. For irreversible actions layer four guardrails, cite the source, cross-check a critical value a second way, run a semantic gate, and put a human or check before the action, so the shape is generate, verify, then act, never generate then act on faith.

Domain 7: Security and Safety

Exam weight: 8%

Section 7A — AI Application Security

Prompt Injection and Jailbreak Defense *(Lessons 100 and 101)*

How prompt injection hijacks an agent through untrusted content, why it differs from a jailbreak, and why the fix lives in your architecture rather than the model's training.

- ▶ Prompt injection is untrusted content that smuggles instructions the model then obeys as commands; direct injection comes from a hostile user, indirect injection rides in through third-party content the model reads, such as a web page, file, or tool result.
- ▶ Injection hijacks what the agent does with untrusted data; a jailbreak targets the model's safety behavior to coax out banned output. A jailbreak is a subset of injection, and jailbreak resistance starts with safety training while injection defense is almost entirely the application's job.
- ▶ Injection is an input problem, so a safer or bigger model does not cure it, and adjusting temperature does nothing. The real defenses are architectural: isolate untrusted input as data, apply least privilege, screen output, and red-team your own agent with hostile documents.
- ▶ For indirect injection, keep third-party content inside tool_result blocks (never the system prompt), JSON-encode it (one layer, never the cure, since the model can still obey plain-language text inside), put your own instructions in the user turn after the result, and screen tool output with a lightweight classifier first.
- ▶ Defense in depth stacks four layers: model, input, and output layers are all probabilistic and cannot stand alone, while the environment layer (sandboxes, filesystem bounds, egress controls) is the only deterministic boundary that holds when every probabilistic layer misses. A policy refusal is the safety layer working, not a bug.

Untrusted Input, Data-Leakage Prevention, and PII Handling *(Lessons 102 and 103)*

Treating every input the application did not author as hostile, and defending sensitive data across the whole lifecycle where it can leak beyond the final reply.

- ▶ Untrusted input is anything the app did not author, including user messages, retrieved documents, web pages, and tool results. Tool and retrieval output is untrusted too: an injection can arrive through a fetched file as easily as through a user message, so it must be inspected and delimited, never trusted as a command.
- ▶ Keep trusted instructions (your system prompt) separate from untrusted data. Deliver third-party content inside tool_result blocks where the model is trained to treat it skeptically, label its source so the model calibrates trust, and delimit or JSON-encode it, though escaping is one layer and never a cure.

- ▶ Sensitive data leaks through more than the visible reply: tool arguments carry raw PII to services that never needed it, logs and traces capture full prompts and payloads, and error messages echo values back out. Map every path a data field takes, not just the response.
- ▶ Shared-context bleed lets one user's data surface in another session; enforce per-user sessions with identity and authorization and never reuse a context window across users. Subagents keep a subtask's data in its own window, which is context hygiene within one session, not the mechanism that separates users.
- ▶ Input minimization (reduce, redact, or tokenize PII before it enters the prompt) is the primary defense because data that never entered cannot leak; output filtering is only a backstop. Keep raw prompts and payloads out of logs, and control egress since approved endpoints can still become exfiltration paths that model-layer inspection never sees.

Section 7B — Guardrails and Safe Deployment

Content Policy and Guardrail Layering *(Lesson 104)*

Why the model's built-in safety is only a floor and how builders layer input, output, and code-level guardrails to enforce their own content policy.

- ▶ Content policy is the set of application-specific rules for what your product will and won't produce or act on. Claude's built-in safety declines broadly harmful requests but cannot know your deployment's bespoke rules, so enforcing what this product refuses is the developer's own guardrail job.
- ▶ A single prompt-level guardrail is probabilistic guidance that a determined user or injected content can talk around, which is fine for low-stakes preferences but not for high-stakes constraints. That is why you layer for defense in depth, so one bypass is caught by the next control.
- ▶ Build three layers: input screening (screen or delimit content before the model sees it, reject or sanitize failures), output checks (validate each response before release to the user or a downstream tool), and code-level enforcement so a rule holds even when the model is bypassed.

Secure-by-Design: Privacy, IAM, Least Privilege *(Lesson 105)*

Building security into an agent's architecture from day one through least privilege and identity-and-access management scoped per user, agent, and tool.

- ▶ Secure by design means applying privacy, identity, and access controls as architectural defaults from the start rather than bolting them on after a breach. Retrofitting fails because injection and leakage paths get designed in before controls exist, a hardening phase filters symptoms not the architecture, and broad early permissions are rarely walked back.
- ▶ Least privilege grants an agent, tool, or identity only the minimum capability it needs and removes everything else, so the capability is absent, not merely watched. If an agent is hijacked, it inherits only what you granted.
- ▶ The least privilege trap is mistaking oversight for a boundary: logging that an agent used a risky capability leaves it in place for the next attacker, and prompting for confirmation is friction, not a boundary. The right fix is to remove the capability so there is nothing to monitor or approve.

Section 7C — Claude Hooks

Hooks as Guardrails: Preventing Destructive Actions *(Lesson 106)*

Using PreToolUse hooks as deterministic guardrails that fire before a dangerous tool call executes, and why the timing of a hook decides whether it can stop damage.

- ▶ A hook is a deterministic script registered in settings.json that fires at a fixed agent lifecycle event, outside the model. It runs the same way every time regardless of the model's output, giving enforcement that a prompt instruction the model can interpret loosely or ignore cannot.
- ▶ PreToolUse fires before a tool call executes, which is the guardrail point: it can deny, ask, or rewrite a pending call, and it is the only tool event that can sanitize a call's arguments before execution. PostToolUse fires after the call has already run, so it can log or react but cannot prevent damage.
- ▶ The permissionDecision field takes exactly four values: allow (proceed as requested), deny (block outright), ask (escalate to a user permission dialog), and defer (hand back to the normal permission flow). Builders commonly remember the first three and forget defer.

Section 7D — Identity, Secrets, and Key Management

Managing Secrets, Credentials, and API Keys *(Lessons 107 and 108)*

Where API keys and other secrets should live, how Claude Code protects and prioritizes them, and why development and production must never share a credential.

- ▶ A secret is any credential your app authenticates with, and one leaked key lets an attacker act as you. Never hardcode or commit a key, keep keys out of prompts and CLAUDE.md files, and load them from environment variables or a secret store; a key in a prompt leaks because the model can echo it, transcripts and request logs capture it, and anyone with log access can read it.
- ▶ Claude Code stores credentials in an OS-protected store: the encrypted system keychain on macOS, a credentials file locked to mode 0600 on Linux, and a credentials file restricted to your account in the user profile on Windows. This file is never a repository artifact, and committing it hands your token to everyone with repo access.
- ▶ Scope every key to least privilege (a read task gets a read-only key, never an admin one) and rotate on a schedule and immediately after any suspected exposure, so a tightly scoped, freshly rotated key is a small prize for an attacker. apiKeyHelper is a shell script that fetches short-lived rotating tokens from a vault, refreshing on a timer or after an auth failure, so you rotate at the source rather than editing code.
- ▶ Development and production always get separate keys: a dev key is low-scope and touches only test data and sandboxes, while a prod key is tightly scoped, rotates from a vault at runtime, reaches real user data, and is never shared back to dev. Separation contains a single leak, gives each environment its own blast radius, and lets you rotate or revoke one without breaking the other.
- ▶ Claude Code follows a defined authentication precedence: cloud provider credentials (Amazon Bedrock, Google Vertex, Foundry) come first, then ANTHROPIC_AUTH_TOKEN for a gateway or proxy, then the direct ANTHROPIC_API_KEY, and below that the apiKeyHelper output and CI OAuth token, so the direct API key is in the middle, not the top. For CI, the setup-token command mints a long-lived OAuth token scoped to inference only, prints it and saves nothing to disk, to be copied into a CLAUDE_CODE_OAUTH_TOKEN variable.

Identity Validation, Access Approval, and Authorized-Access Monitoring (Lesson 109)

The full access-control sequence for a privileged agent action: prove identity, authorize each action, approve the risky ones, then monitor the trail.

- ▶ Authentication verifies WHO a caller is through a login, token, or key, while authorization determines WHAT that verified identity may do, checked on every requested action. An agent needs both, and treating one as if it covered the other is the most common access mistake.
- ▶ A valid login is not access; being authenticated proves identity, not entitlement to an action. Enforce per-action authorization on every sensitive call, and run that check at the downstream system that actually holds the data or capability, not by trusting the model to police itself.
- ▶ Access approval puts a human in the loop for high-stakes operations such as deleting data, moving money, or changing access. Gate the action first, get approval, then execute, because approval is enforcement that happens before the action, not a note recorded after it.

Domain 8: Tools and MCPs

Exam weight: 11%

Section 8A — Tool Implementation

Tool Descriptions and Schemas *(Lessons 110 to 112)*

Why the description, not the name, is what Claude routes on, and how the schema and tool-use loop turn a definition into a valid call.

- ▶ A tool definition has three fields: name (the identifier used to invoke and match the call, matching a pattern of letters, digits, underscores, and hyphens), description (the primary routing signal), and input_schema (a JSON Schema declaring parameter types and which fields are required).
- ▶ The description is by far the most important factor in tool performance; a strong one covers four things, what the tool does, when to use it, when NOT to use it, and caveats or limits. Write three to four sentences like onboarding notes, and when Claude skips or misroutes a tool, rewrite the description first, before touching temperature or the model.
- ▶ input_schema does three jobs: it declares each parameter, types it (string, integer, boolean, array, or object), and lists the required fields. Reserve required for fields with no safe default such as a record ID, since over-requiring makes Claude stop and ask the user; use enums for fixed value sets, and setting strict true uses grammar-constrained sampling to guarantee arguments match the schema exactly.
- ▶ A tool call is a four-step round trip: you send the request with your tool definitions, Claude returns stop_reason tool_use with the input, your code runs the tool and returns a tool_result (linked by tool_use_id), then Claude answers. tool_choice has four values: auto (default when tools are provided), any (some tool), tool (one named tool), and none (default when no tools are provided).
- ▶ bash, text_editor, memory, and computer_use ship with Anthropic-defined schemas but are still client tools your own code executes, pre-typed does not mean server-run. Arguments come back as a JSON object you parse, but a schema-valid call is never guaranteed to succeed at runtime; for complex parameters, a schema-valid input example lifts accuracy more than extra prose.

Tool Errors and Usage Patterns *(Lessons 113 and 114)*

The split between client-side and server-side tool execution, and how to handle failures, gate risky calls, and let the model dispatch tools.

- ▶ Client tools execute in your application, Claude returns stop_reason tool_use and your code runs the call and returns a tool_result. Server tools run on Anthropic's infrastructure with no handler and no tool_result, since the result returns inline. bash, text_editor, memory, and computer_use are client tools; web_search, web_fetch, code_execution, and tool_search are server tools.
- ▶ One case blurs the line: a server tool grouped in parallel with a client tool falls back to stop_reason tool_use, so your loop drives every call in that mixed batch.

- ▶ Return tool failures instead of crashing the loop, put the error text in the `tool_result` content and set `is_error` to true so Claude can retry or adapt. Server tool errors are caught transparently by Claude, so you never set `is_error` for them, only client tool failures come back to your code.
- ▶ Message-format rules cause silent 400 errors: a `tool_result` must immediately follow its `tool_use`, `tool_result` blocks come first in a user turn before any text, and there is no separate tool role, results ride inside user messages.
- ▶ In agentic dispatch the model picks each tool from the request and tool descriptions rather than a fixed if-then branch, trading determinism for flexibility. Gate high-impact calls such as deleting data or sending money in your runtime or an Agent SDK `PreToolUse` hook, never rely on Claude to self-police them.

Tool Set Construction Best Practices *(Lessons 115 and 116)*

How to design a tool set the model routes cleanly through: fewer well-scoped tools, distinct names, and pruning weighed against per-tool token cost.

- ▶ A tool set is a menu the model reads every turn, so fewer, well-scoped tools reduce selection ambiguity and each should map to one clear action. You cannot judge a set by reading it, prove it with realistic multi-tool evaluations that track accuracy, tool-call count, tokens, and runtime, then read the transcripts to find ambiguous descriptions.
- ▶ Design tools around agent workflows, not one per API endpoint, which floods the menu with near-duplicates. Consolidate related actions on the same resource behind one parameterized tool, for example a single `manage-pull-request` tool with an `action` parameter instead of separate `create`, `review`, and `merge` tools.
- ▶ Give every tool a distinct, action-oriented name and namespace with the service when tools span systems (`billing_list_invoices`, not a bare `list`). Misrouting traces to three sources: similar names, overlapping descriptions, and vague scope, and the fix for all three is to sharpen names, separate descriptions, and tighten scope.
- ▶ Every provided tool costs tokens: on Opus 4.8 providing tools adds roughly 290 tokens of overhead plus the names, descriptions, and schemas, and even `tool_choice none` pays it, the only zero-token case is providing no tools at all. Prune a tool that evals show is rarely the right call, is redundant with another, or needlessly widens the attack surface.
- ▶ For large sets, the Tool Search Tool lets Claude discover tools on demand with rarely-used tools marked `defer_loading true` while the most-used stay loaded; programmatic calling keeps intermediate results out of context (first-party API only); reserve an MCP server for capabilities reused across many clients, the deciding factor is shared reuse, not raw tool count.

Section 8B — MCP Server Development

MCP Server Authoring *(Lessons 117 and 118)*

The three MCP primitives, the controller each answers to, and how a server registers and serves them.

- ▶ An MCP server exposes three primitives and ownership is the distinction that matters most: tools are model-controlled (the model discovers and invokes them), resources are application-controlled read-

only data (the host injects them, the model never fetches), and prompts are user-controlled templates (a person selects them, often surfaced as slash commands).

- ▶ Each resource is identified by a URI and only supplies data, it never performs an action; tools are the sole primitive that acts, wrapping an API call, a database query, or a file operation.
- ▶ Registration and retrieval each have their own methods: tools register with a name, description, and inputSchema and run via tools/call; resources register with a URI and are read via resources/read; prompts register with a name and optional arguments and are fetched via prompts/get. Every primitive also offers a list method (tools/list, resources/list, prompts/list) so a client can discover what the server exposes before using it.
- ▶ Mind the spelling layer: MCP uses camelCase on its JSON-RPC wire format (inputSchema, isError, tools/call), while the Claude Messages API uses snake_case (input_schema, is_error, tool_use), same ideas, different layer, so do not conflate them.
- ▶ Roots name the filesystem paths or URIs a client puts in scope but are only advisory, a well-behaved server should respect them yet a local stdio server still runs with its OS user's full file access, so real isolation comes from OS sandboxing, containers, and permissions. MCP is an open, model-agnostic standard, so one server can serve many compliant clients.

MCP Communication Patterns *(Lessons 119 and 120)*

MCP's two standard transports, local stdio and networked Streamable HTTP, and the OAuth flow that authenticates a remote server's clients.

- ▶ MCP separates a data layer (the JSON-RPC 2.0 protocol defining the lifecycle, primitives, and exact message shape) from a transport layer (the channel that carries the messages); stdio and Streamable HTTP are the two standard transports. The host runs one client per server, and each client holds exactly one connection to one server.
- ▶ stdio runs the server as a local child process the client launches and manages: the client writes newline-delimited JSON-RPC to stdin, the server replies on stdout (protocol messages only, one per line), and logs go to stderr. It fits local, single-user, zero-network setups and reads credentials from the environment, never OAuth.
- ▶ Streamable HTTP, introduced in 2025, is the remote transport: an independent server serving many clients through a single endpoint that accepts both POST and GET, with optional event-stream replies. It replaced the deprecated 2024 HTTP plus Server-Sent Events transport, which new servers should not build on.
- ▶ Two headers keep an HTTP session coherent: Mcp-Session-Id (assigned by the server on the initialize response, then echoed by the client on every later request) and MCP-Protocol-Version (so both sides agree which protocol revision they speak). The server also validates the Origin header against DNS rebinding, binds to 127.0.0.1 when running locally, and authenticates clients before serving any tools.
- ▶ Networked auth is OAuth 2.1, defined for HTTP only and never for stdio, with three roles: the resource server (the MCP server, validates the token on every request), the OAuth client (obtains and attaches the token), and the authorization server (issues tokens, co-hosted or independent). The access token rides in the Authorization header as a bearer token, never in the query string, and the server confirms the token's audience matches this server.

Section 8C — Agentic Customization

Choosing Among Built-in Tools, Custom Tools, Skills, and MCPs *(Lessons 121 and 122)*

The four ways to extend Claude and how to choose the lightest mechanism that fits each capability.

- ▶ Four extension mechanisms exist: built-in (server) tools run on Anthropic's infrastructure with no handler (web search, web fetch, code execution, tool search) but cannot reach private systems; custom tools are functions you define and run yourself, reaching anything your code can; Skills package reusable procedural know-how in a SKILL.md file loaded on demand; MCP servers expose tools, resources, and prompts over a standard, reusable interface.
- ▶ A Skill is a folder with a SKILL.md whose frontmatter carries a name and description; Claude reads only that short description each turn and loads the full Skill when the task matches, which is progressive disclosure that keeps its bulk out of context. On the Claude API a Skill makes no external network calls, so it pairs with a tool or MCP server when it needs live data.
- ▶ The guiding rule is to pick the lightest mechanism that does the job, because each one adds context cost or infrastructure: try a built-in first, then a custom tool, and only then a Skill or MCP; reaching for an MCP server when a custom tool fits is overbuilding.
- ▶ Three questions resolve most decisions: if Anthropic already hosts the capability, use the built-in and stop; if the need is specific to a single application, a custom tool is the right size; if a capability is shared across clients or teams, reach for a Skill (for procedure) or an MCP server (for system access). The three forces underneath are reach into private systems, reuse scope, and context cost.
- ▶ Skills and MCP are complementary, not rivals you choose between: the MCP server brings the reach to external systems while the Skill brings the method, which tool to call and in what order. The framework is applied per capability, so a single agent often combines a built-in tool, a custom tool, an MCP server, and a Skill at once.